

Resumen de Conceptos Importantes del Curso de Spring Boot (30 Videos)

I. Introducción, Arquitectura y Herramientas

Concepto	Descripción	Fuentes Clave
Spring Framework vs. Spring Boot	Spring Framework es un marco de trabajo (<i>Framework</i>) para la plataforma Java que proporciona una infraestructura integral y flexible para simplificar el desarrollo de aplicaciones empresariales. Spring Boot es una extensión de Spring Framework que simplifica drásticamente el desarrollo de aplicaciones al proporcionar autoconfiguración y gestión de dependencias integradas. Al utilizar Spring Boot, se está utilizando implícitamente Spring Framework.	
Herramientas Esenciales	Se requiere el JDK 17 o superior para desarrollar aplicaciones con Spring Boot. El Entorno de Desarrollo Integrado (IDE) recomendado es IntelliJ IDEA . Se utiliza Postman como herramienta de desarrollo para probar, crear y documentar las API REST.	
Arquitectura de Microservicios	Es una arquitectura de software que descompone una aplicación monolítica en servicios pequeños e independientes. Una ventaja clave es la tolerancia a fallos , ya que el fallo de un servicio no afecta necesariamente a otros.	
Arquitectura Basada en Capas	Enfoque que se puede aplicar a un microservicio para segmentar responsabilidades en distintas capas: Presentación (<i>controllers</i>), Lógica de Negocio (<i>domain, service, exceptions</i>) y Acceso a Datos (<i>persistence, entities</i>).	
API REST	API es un conjunto de protocolos que facilitan la comunicación entre componentes de software. API REST es un estilo arquitectónico basado en estándares web que utiliza métodos HTTP para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre recursos identificados por URLs.	
Estructuración y Versionado de URL	La convención de URL estructurada y versionada debe incluir: Protocolo (HTTP) > Nombre del servidor > Puerto > Context Path (ej., <i>/sistema</i>) > API Path (ej., <i>/api</i>) > Versión (ej., <i>/v1</i>) > Recurso (ej., <i>/clientes</i>). El versionado en la URL (ej., <i>v1</i>) es el método más común.	

Archivos de Configuración El archivo predefinido de configuración es **application.properties**, donde se pueden modificar configuraciones como el puerto (**server.port=8080**) o el contexto de la aplicación (**server.servlet.context-path**). También puede usarse **application.yml**.

II. Controladores, Mapeos y Operaciones CRUD

Anotación/Concepto	Descripción	Fuentes Clave
@SpringBootApplication	Anotación principal que marca la clase de arranque de la aplicación Spring Boot.	
@RestController	Marca una clase como un controlador REST, combinando la funcionalidad de @Controller y @ResponseBody (devuelve datos directamente en JSON/XML).	
@RequestMapping	Utilizada a nivel de clase para proporcionar una ruta base común (prefijo) a todos los <i>endpoints</i> del controlador, unificando las URLs.	
@GetMapping	Mapea solicitudes HTTP GET (lectura) a un método específico.	
@PostMapping	Mapea solicitudes HTTP POST (creación) a un método específico.	
@PutMapping	Mapea solicitudes HTTP PUT para actualizar o reemplazar por completo un recurso.	
@PatchMapping	Mapea solicitudes HTTP PATCH para aplicar modificaciones parciales a campos específicos, sin necesidad de enviar la representación completa del recurso.	
@DeleteMapping	Mapea solicitudes HTTP DELETE para eliminar un recurso.	
@PathVariable	Se usa para mapear valores dinámicos que vienen en la URL (dentro de llaves { }) a parámetros del método.	
@RequestBody	Indica que un parámetro del método debe vincularse al valor del cuerpo de la solicitud HTTP (generalmente JSON).	

POJO (Plain Old Java Object)	Clase Java básica utilizada para representar la estructura de una entidad (atributos, constructor, <i>getters</i> y <i>setters</i>). Las clases POJO sirven para transferir información.
-------------------------------------	---

III. Gestión de Respuestas HTTP y Eficiencia

Concepto	Descripción	Fuentes Clave
Clase <code>ResponseEntity</code>	Clase <i>wrapper</i> (envoltorio) que encapsula un objeto y representa la respuesta HTTP completa, permitiendo controlar el cuerpo, los encabezados y el código de estado . Usar <code>ResponseEntity<?></code> permite que el tipo de respuesta (cuerpo) varíe (ej., objeto o mensaje de error <code>String</code>).	
Códigos de Estatus (Respuestas REST)	200 OK : Solicitud exitosa. 201 Created : Recurso creado exitosamente (típico de POST). 204 No Content : Solicitud exitosa sin contenido (típico de PUT/DELETE; no es necesario retornar el recurso o un mensaje). 404 Not Found : Recurso no encontrado.	
Creación Eficiente de Recursos	Es una buena práctica retornar el código 201 Created junto con el encabezado Location , el cual indica la URI/URL del recurso recién creado. La clase <code>ServletUriComponentsBuilder</code> se utiliza para construir esta URI de manera programática.	
URI vs. URL	Una URI (<i>Uniform Resource Identifier</i>) es una cadena de texto que identifica un recurso de manera única. Una URL (<i>Uniform Resource Locator</i>) es un tipo específico de URI que no solo identifica el recurso, sino que también proporciona la forma de localizarlo. Todas las URL son URI, pero no todas las URI son URL.	

IV. Inyección de Dependencias (DI) y Control de Beans

Concepto	Descripción	Fuentes Clave
Programación Orientada a Interfaces	Enfoque que define el comportamiento con Interfaces (contratos). Una clase implementadora cumple el contrato. Esto promueve la flexibilidad, abstracción y escalabilidad .	
Inversión de Control (IoC)	Principio en el que se basa la DI. El control de la creación y gestión de objetos (Beans) se delega a un contenedor o <i>framework</i> , como Spring .	
Bean	Un objeto gestionado por el contenedor de Spring. Las clases decoradas con anotaciones estereotipadas (ej., <code>@RestController</code> , <code>@Service</code>) se convierten en Beans.	
Inyección de Dependencias (DI)	Patrón de diseño para recibir dependencias desde el exterior, promoviendo bajo acoplamiento. La inyección por campo se realiza en los atributos de la clase.	
<code>@Service</code>	Anotación que indica a Spring que una clase debe ser tratada como un Bean de servicio que encapsula la lógica de negocio.	
<code>@Autowired</code>	Anotación fundamental para DI. Le indica al contenedor de Spring que inyecte automáticamente una instancia del Bean requerido en el campo, constructor o método.	
Estrategia <code>@Primary</code>	Se usa para resolver conflictos cuando hay múltiples Beans del mismo tipo, indicando cuál debe ser el preferido o predeterminado .	
Estrategia <code>@Qualifier</code>	Se usa para especificar exactamente qué Bean inyectar, utilizando el nombre asignado al servicio (ej., <code>@Service("nombre")</code>). <code>@Qualifier</code> tiene prioridad sobre <code>@Primary</code> .	

Estrategia @ConditionalOnProperty	Permite activar o desactivar Beans solo si se cumple una condición basada en el valor de una propiedad externa (ej., en <code>application.properties</code>).
Jackson (Serialización/Deserialización)	Biblioteca para el procesamiento de datos JSON en Java. La Deserialización (JSON a objeto Java) requiere que la clase POJO tenga un constructor por defecto (vacío) .
Carga Diferida (@Lazy)	Permite diferir la creación de la instancia de un Bean hasta que sea realmente necesario, optimizando el tiempo de arranque de la aplicación. Debe usarse tanto en el Bean como en el controlador que lo inyecta.
Externalización de Configuración	Proceso de separar los valores de configuración (claves, credenciales, metadatos) del código fuente. Se logra mediante: @Configuration (indica que la clase es fuente de configuración) y @ConfigurationProperties (mapea propiedades externas basadas en un prefijo, ej., <code>prefix="app"</code>) a los atributos de una clase POJO.
